

Grado en Ciencia e Ingeniería de Datos

PROCESAMIENTO DEL LENGUAJE NATURAL

Práctica 1: Preparación y procesamiento de textos

Comienzo	martes 15 de septiembre de 2026
Finalización	martes 20 de octubre de 2026 (hasta las 15:50h)

OBJETIVOS

Esta primera práctica de la asignatura tiene dos objetivos principales:

- **Aprender a adquirir y preparar contenidos textuales para su procesamiento**, usando los estándares de lectura y procesamiento de la librería NLTK.
- **Aprender a preprocesar textos** para una tarea/aplicación de procesamiento del lenguaje natural particular siguiendo un pipeline reutilizable y extensible.

Para ello, de forma gradual, se codificará un proyecto en Python que harán uso de librerías de gestión de corpus de texto y preprocesamiento de texto.

El conjunto de textos procesados resultantes y la arquitectura implementada en esta práctica serán empleados en las prácticas 2 y 3 de la asignatura.

CASO DE USO

El propósito final de las prácticas de la asignatura se enfoca en la clasificación automática de textos. En particular, la clasificación de reseñas generadas por usuarios (user reviews) atendiendo a la polaridad de sus opiniones subyacentes, p. ej., positivas, neutras o negativas.

Por ello, es necesario disponer de un corpus anotado que permita construir y evaluar modelos de clasificación diversos. Dicho corpus tendrá que estar compuesto de un número relativamente grande de textos con valoraciones personales dadas por usuarios sobre determinados ítems, así como con etiquetas que indiquen la polaridad de opinión global de tales valoraciones.

En particular, en la asignatura se plantea construir y emplear un **corpus con reseñas en inglés sobre productos de Amazon**, disponibles públicamente a través del dataset publicado en la siguiente web: <https://amazon-reviews-2023.github.io/>

En este dataset, encontraremos un total de 571 millones de reviews que han hecho más de 54 millones de usuarios a más de 48 millones de productos. Sobre los productos, el dataset proporciona un objeto JSON por cada uno de ellos con información relevante sobre cada producto: descripción, título, categorías, características, etc. Un ejemplo de descripción de un ítem se puede encontrar en el siguiente enlace:

<https://amazon-reviews-2023.github.io/#load-item-metadata>

Además, el dataset ofrece las reviews que han hecho los usuarios a cada uno de los productos. De una review se puede extraer: el id del usuario, la valoración del usuario de 1 a 5, los comentarios del usuario en texto libre, si es una compra verificada o cuando ha hecho la review entre otros. Un ejemplo del objeto que contiene la review se puede ver en el siguiente enlace:

<https://amazon-reviews-2023.github.io/#load-user-reviews>

Este dataset también divide los productos y las reseñas por categorías de productos, siguiendo el mismo esquema de la página web de Amazon. Se pueden consultar los grupos y el número de productos y reviews que contienen en <https://amazon-reviews-2023.github.io/#grouped-by-category>.

Para esta práctica se propone que selecciones un conjunto de las categorías para incorporar en el corpus de las prácticas. Puede ser solo una muy extensa o una combinación de algunas.

ESTRUCTURA

La práctica consta de los siguientes ejercicios a realizar en el orden establecido:

1. Descargar y preparar la información de los productos y sus reseñas.
2. Diseñar e implementar el gestor del corpus.
3. Preprocesamiento básico del corpus y análisis del corpus final.

A continuación, se proporciona el enunciado de cada uno de estos ejercicios.

EJERCICIO 1: Descargar y preparar la información de los productos y sus reseñas

En este primer ejercicio se pretende preparar el almacenamiento en el disco local del corpus de reseñas de productos de Amazon. Para ello, aplicaremos un método que siguen muchos corpus aplicados a [NLTK](#) para que puedan ser utilizados en la propia librería de manera eficiente.

El método más común es almacenar los textos en documentos individuales en un sistema de ficheros dentro del disco. Para ello, vamos a hacer una estructura de directorios y carpetas que sea compatible con el formato que espera la clase [CategorizedCorpusReader](#). El objetivo es organizar los documentos por carpetas según su categoría, por ejemplo, por opiniones positivas, negativas o neutrales. Esto nos permitirá obtener textos, oraciones o palabras y tokens de cada una de estas categorías individualmente.

A parte de esta estructura de los ficheros, es necesario definir el formato en el que almacenamos la información. Este formato es flexible, es decir, los estudiantes pueden elegir almacenar la información en texto libre, objetos JSON, etc. Sin embargo, esta elección hará que, a la hora de implementar el `CorpusReader` concreto, se deba tener en cuenta.

Para este ejercicio, se pide desarrollar y entregar un script `pln_p1_XXXX_YY_e1.py` que realiza las siguientes operaciones:

- Descargue los ficheros de reviews necesarios para el corpus elegido desde la web del dataset de Amazon.
- Extraiga la información necesaria del corpus (reviews y metadatos) y que los almacene siguiendo la estructura de carpetas que permita definir las categorías.

EJERCICIO 2: Diseñar e implementar el gestor del corpus

Una vez que tenemos la estructura de los ficheros creada correctamente, lo siguiente será implementar una clase que permita extraer la información de este corpus de manera efectiva.

Esta clase debe heredar de dos clases principales que hay dentro de NLTK: [`CorpusReader`](#) y [`CategorizedCorpusReader`](#).

`CorpusReader` es una clase abstracta que permite definir la infraestructura de gestión de los textos en NLTK. Entre sus tareas se encuentra: resolver y filtrar archivos por el `fileid`, rastrear el sistema de archivos de forma recursiva y resolver las rutas a los ficheros, gestionar la carga perezosa del corpus (no cargar todo el corpus, solo leemos según lo vamos necesitando), abstracciones y gestión de las codificaciones de los ficheros.

Por otro lado, la clase `CategorizedCorpusReader` tiene como objetivo incluir a un lector de corpus la generación de etiquetas o categorías. Esta definición de categorías se puede hacer de diversas maneras: con expresión regular al nombre del fichero, por un diccionario directo o a partir de metadatos. Además, permite implementar los métodos que nos van a permitir a navegar entre los textos de una categoría, o las categorías de un texto.

En este ejercicio se pide desarrollar y entregar la clase `AmazonCorpusReader` que permita leer los textos del corpus, realizando las siguientes operaciones:

- Implementar la clase usando la herencia con `CategorizedCorpusReader` y `CorpusReader` para seguir el estándar de NLTK.
- Obtener los ficheros, tanto por categorías, como por textos individuales.
- Devolver el contenido de los documentos.
- Devolver el rating de cada una de las reviews.
- Devolver el texto, párrafos, oraciones y palabras individuales de todos los textos.

EJERCICIO 3: Preprocesamiento básico del corpus y análisis básico

Después de poder acceder a nuestro corpus a distintos niveles, el último paso es incluir las tareas de preprocesamiento. Para este último ejercicio, se pide modificar las funciones necesarias en la clase `AmazonCorpusReader` creada en el ejercicio 2 para que:

- Se hagan tareas de preprocesamiento de textos en el lugar que correspondan:
 - Eliminación de etiquetas HTML si las hubiera.
 - Eliminación párrafos, oraciones o palabras en blanco, si las hubiera.
 - Tratamiento o filtrado de etiquetas de marcado, símbolos especiales, signos de interrogación y exclamación, caracteres repetidos de forma consecutiva, emojis, etc.
 - Filtrado de stopwords, aunque se recomienda mantener registro del texto con stopwords para posibles procesamientos de más alto nivel.
- Implementar una función que permita obtener una descripción del corpus: nº documentos, tamaño total, vocabulario, etc.

Los análisis gramatical (PoS tagging) y sintáctico (constituency/dependency parsing) se posponen para la práctica 2. Las operaciones de stemming y lemmatization tampoco se consideran prioritarias en esta práctica, pudiendo ser realizadas en prácticas siguientes si fuera necesario.

MEMORIA

Se deberá elaborar un documento de memoria `pln_p1_XXXX_YY_memoria.pdf` concisa que incluya:

- Nombres y apellidos de los autores.
- Listado de operaciones de preprocesamiento de texto implementadas.
- Descripción de la estructura, organización y funciones de acceso del corpus desarrollado.
- Estadísticas del corpus: número de reseñas, número de reseñas con rating, número de reseñas con texto, número de reseñas con rating y texto, distribución de ratings, número de productos, número de usuarios, etc.

ENTREGA

Como resultado de la práctica, cada equipo deberá entregar una carpeta comprimida en un fichero comprimido ZIP. El nombre de la carpeta y del fichero ZIP será `pln_p1_XXXX_YY.zip`, donde XXXX ha de sustituirse por el código del grupo de prácticas (7461 o 7462), e YY ha de sustituirse por el identificador de equipo: 01, 02, etc.

El fichero ZIP contendrá:

- El código Python (carpeta `src`) de los programas desarrollados en los tres ejercicios.
- El corpus construido o parte de él para limitar el tamaño del ZIP a entregar.
- La memoria elaborada.

PLANIFICACIÓN SUGERIDA

Se propone la siguiente planificación de trabajo:

- **1ª semana**
 - Realizar el ejercicio 1.
- **2ª semana**
 - Realizar el ejercicio 2.
- **3ª semana**
 - Realizar el ejercicio 3.
- **4ª semana**
 - Finalizar la realización del ejercicio 3.
 - Redactar los apartados de preprocesamiento de texto y creación del corpus en la memoria.

BIBLIOGRAFÍA SUGERIDA

Para esta práctica, se pueden seguir los capítulos 2 y 3 del libro *Applied Text Analysis with Python*, accesible como recurso online desde la biblioteca digital de la UAM:

- [Acceso al libro.](#)